

Ngenux Decision Automation Engine (DAE)

Intern Program Handbook

Version 1.2

Duration: 8 Weeks

Delivery Model: AWS-first, weekly Friday demos

Code Quality: PRs, Reviews, SonarLint, Tests mandatory

1. Program Vision

This intern program builds a real Ngenux SKU: the Decision Automation Engine (DAE). Interns will work on a production-grade system that evaluates policy-driven decisions using explicit inputs and produces explainable, auditable outputs.

The objective is not learning in isolation, but delivering a system that is deployable on AWS, reviewed through proper engineering workflows, and usable in sales demos and pilots.

2. SKU Overview

The Ngenux Decision Automation Engine (DAE) is a reusable decision runtime that converts decision requests, controlled context data, policy references, and actor information into structured, explainable outcomes. It supports policy versioning, agent orchestration, human overrides, observability, and audit trails, with a lightweight Streamlit UI for demos and operations.

3. System Contract (Input / Output)

Input to the system includes:

- Decision request (decision_type, request_id)
- Actor information (id, role, source)
- Policy reference (policy_id, optional version)
- Context envelope with structured case facts

Output from the system includes:

- decision_id and execution status
- structured decision output
- policy version used
- structured explanation
- latency and cost metadata

4. Program Rules

1. Week 1–2 focus on onboarding and local development; AWS deployments begin Week 3.
2. From Week 3 onward, all Friday demos must run from AWS.
3. All code changes must go through GitHub pull requests and reviews.
4. SonarLint issues and failing tests block PR merges.
5. If a feature cannot be demoed on Friday, it is incomplete.

Week 1: Onboarding & Engineering Foundations

Weekly Objective:

Align interns to the product vision, engineering standards, and development workflows required to build a production-grade Ngenux SKU. No customer-facing features are expected this week.

Learning Objectives:

Understand the product, GitHub workflows, PR reviews, basic testing, static analysis, and AWS fundamentals.

Concepts Covered:

GitHub flow, PR reviews, SonarLint, test fundamentals, AWS mental model.

Daily Execution Plan:

Day 1: Program kickoff, product walkthrough, system contract explanation, repo walkthrough.

Day 2: GitHub branching strategy, commit conventions, PR creation and review process.

Day 3: SonarLint setup, understanding common code smells, fixing sample issues.

Day 4: Testing basics: unit vs integration tests, writing sample test cases.

Day 5: AWS overview: ECS, RDS, IAM concepts, how Friday demos will work on AWS.

Friday Demo:

Walkthrough of product vision, system contract, GitHub workflow, and AWS architecture (no live system demo).

Week 2: Core Runtime & UI (Local, Production Discipline)

Weekly Objective:

Build the first working slice of the system locally while strictly following PR reviews, static analysis, and test coverage expectations.

Learning Objectives:

Apply API-first design, persistence, audit logging, PR-based collaboration, and testing practices.

Concepts Covered:

FastAPI, Postgres, Alembic, Streamlit, PR reviews, SonarLint, unit tests.

Daily Execution Plan:

Day 6: Set up FastAPI app, health endpoint, logging, and project structure.

Day 7: Configure Postgres and Alembic; create decisions and audit_logs tables.

Day 8: Implement decision execution endpoint with persistence and audit logging.

Day 9: Write unit tests for decision execution and DB persistence.

Day 10: Build Streamlit UI (execute, list, detail) and merge via reviewed PRs.

Friday Demo:

Execute a decision locally, show persisted data, passing tests, clean SonarLint, and reviewed PRs.

Week 3: AWS Deployment & Policy Store

Weekly Objective:

Deploy the system to AWS and introduce versioned policies so decisions are governed by explicit rules.

Learning Objectives:

Learn containerization, managed infrastructure, secrets handling, and policy-as-data.

Concepts Covered:

Docker, ECR, ECS Fargate, ALB, RDS, Secrets Manager, policy versioning.

Daily Execution Plan:

Day 11: Create Dockerfiles, ECR repositories, and push images.

Day 12: Provision RDS Postgres and Secrets Manager; configure networking.

Day 13: Deploy API to ECS behind ALB and verify health.

Day 14: Deploy Streamlit UI to ECS and connect to API.

Day 15: Implement policy and policy_version tables and CRUD APIs.

Friday Demo:

Run the system from AWS, create a policy, execute a decision using that policy, and verify data in RDS.

Week 4: Replay & Explainability

Weekly Objective:

Ensure decisions are reproducible and transparent through replay and structured explanations.

Learning Objectives:

Understand determinism, rule tracing, and explanation design.

Concepts Covered:

Policy binding, replay mechanics, explanation schemas.

Daily Execution Plan:

Day 16: Bind policy versions to decisions for historical accuracy.

Day 17: Implement deterministic decision replay endpoint.

Day 18: Define explanation schema and capture rule evaluation traces.

Day 19: Persist explanations and include them in API responses.

Day 20: Enhance UI to support replay and explanation views.

Friday Demo:

Execute and replay a decision on AWS and walk through the explanation in the UI.

Week 5: Context Control

Weekly Objective:

Control and audit what data is passed into the decision engine.

Learning Objectives:

Learn least-privilege data access and redaction.

Concepts Covered:

Context envelopes, validation, redaction, audit events.

Daily Execution Plan:

Day 21: Define context schema per decision type.

Day 22: Context validation and rejection.

Day 23: Field-level redaction rules.

Day 24: Context usage and violation logging.

Day 25: UI view for context usage.

Friday Demo:

AWS demo: show redaction, context violations, and audit logs.

Week 6: Agent Orchestration

Weekly Objective:

Modularize decision logic using controlled multi-agent execution.

Learning Objectives:

Understand orchestrator patterns and agent boundaries.

Concepts Covered:

Orchestrator, policy agent, compliance agent, execution traces.

Daily Execution Plan:

Day 26: Implement orchestrator flow.

Day 27: Policy agent implementation.

Day 28: Compliance agent implementation.

Day 29: Agent step tracing.

Day 30: UI timeline for agent execution.

Friday Demo:

AWS demo: show agent order, timing, and failure handling.

Week 7: RBAC, Observability & Cost

Weekly Objective:

Make the system enterprise-operable by adding human overrides, visibility into performance, and cost awareness.

Learning Objectives:

Understand RBAC, human-in-the-loop workflows, metrics, and cost attribution.

Concepts Covered:

Roles and permissions, overrides, decision locking, metrics, dashboards.

Daily Execution Plan:

Day 31: Define user roles and enforce permissions at the API layer.

Day 32: Implement manual overrides with audit capture and decision locking.

Day 33: Add annotations and before/after decision views.

Day 34: Implement latency, error, and cost tracking per decision.

Day 35: Build metrics and cost dashboards in Streamlit.

Friday Demo:

Override a decision on AWS, show audit trail, and demonstrate live metrics and cost updates.

Week 8: Refund Demo, Performance Testing & SKU Packaging

Weekly Objective:

Finalize the system as a sales-ready SKU and validate performance under load.

Learning Objectives:

Learn verticalization, performance testing, and demo packaging.

Concepts Covered:

Refund policies, Locust load testing, demo scripting.

Daily Execution Plan:

Day 36: Create refund policy pack and predefined demo scenarios.

Day 37: Wire refund-specific inputs and flows into the engine.

Day 38: Build guided refund demo UI.

Day 39: Create Locust performance tests and run load against AWS.

Day 40: Finalize documentation, architecture diagram, and demo script.

Friday Demo:

Run full refund demo from AWS and show Locust performance results.